Project Report

Assessment - Threat score system design & implementation

Project Acronym:	Assessment
Project Title:	Assessment - Prohibited item / weapon detection
Candidate Name / Lead Engineer:	M. R. S. C Karunarathne
Target Role:	Applied Machine Learning Engineer
Date of Submission:	14 June 2026

Executive Summary

This report details the end-to-end design, implementation, and evaluation of an automated threat scoring system, developed for the Applied ML Engineer assessment. Focusing on the domain of weapon detection in scene-understanding applications, the objective was to bridge the gap between raw computer vision metrics and actionable security intelligence. The system utilizes a publicly available Kaggle dataset comprising over 27,000 bounding-box annotations. To satisfy the constraint for lightweight, edge-deployable solutions, the YOLOv8 nano architecture was established as the baseline. Initial training runs highlighted severe infrastructure memory bottlenecks (Kaggle out-of-memory failures), which were successfully mitigated through custom data loader pipeline . The baseline model achieved an initial mean Average Precision (mAP@50) of 0.4647.

To elevate performance without increasing inference latency, advanced hyperparameter optimization was applied to the baseline, transitioning the optimizer from SGD to AdamW with a Cosine Annealing learning rate schedule over an extended 80-epoch horizon. This resulted in a definitive +3.04% increase in mAP@50 and a highly critical +4% boost in overall Precision, significantly reducing false-positive alert fatigue for potential human operators. Because the dataset relies on a single "Weapon" class taxonomy, threat severity could not be determined by weapon type. Therefore, a custom Threat Score heuristic was engineered to translate the optimized machine learning outputs into business logic. This deterministic formula combines prediction confidence, spatial center-proximity, and bounding-box scale—acting as a geometric proxy for physical distance to the camera. The final output translates these signals into a 0-100 severity scale segmented into actionable alert tiers, demonstrating a complete pipeline from raw pixel data to deployable operational intelligence.

Table of Contents

2 Description of Project Context and Objectives

2.1 Threat Domain Justification

The deployment of automated weapon detection systems represents one of the most operationally demanding use cases in the computer vision area. This domain was selected because it rigorously tests the ability to balance raw algorithmic accuracy with severe edge-compute latency constraints. Architecting a public safety pipeline requires following strict security compliance standards principles that are just as critical here as they are when deploying dynamic liveness detection in secure access systems. The system cannot afford to be passive; it must operate dynamically to guarantee immediate alert escalation without vulnerability to spoofing or environmental noise.

Justification for selecting this domain is based on three core operational challenges:

- *Asymmetric Risk Profile (The Cost of Error):* In weapon detection, the cost of a False Negative (failing to detect a drawn firearm) is catastrophic. On the other hand, a high False Positive rate (flagging cellphones, tools, or umbrellas) induces "alert fatigue" in human operators, rendering the security system useless. This severe asymmetry dictates that raw bounding-box predictions are insufficient, necessitating the development of a highly calibrated downstream Threat Scoring engine.
- *Stringent Sub-30ms Latency Budgets:* Unlike passive hazard monitoring (ex: unattended baggage), which can afford multi-second polling intervals, an active weapon threat is an immediate kinetic event. The architecture must sustain real-time processing (min 30 FPS) on edge-constrained hardware, strictly prohibiting the use of high-latency, cloud-dependent foundation models.
- *Complex Visual Morphology and Occlusion:* Handheld weapons frequently occupy a minuscule percentage of the overall pixel space in a standard wide-angle CCTV frame. Furthermore, they are subject to extreme variance in lighting, motion blur, and physical occlusion by human hands or clothing.

To formalize the boundaries of this project, the following architectural baseline was established to guide the ML development lifecycle and justify the subsequent model selection:

Deployment Parameter	Operational Target	ML Pipeline Implication
Inference latency	< 30ms per frame	the selection of edge-optimized nano-architectures (YOLOv8n)
Security Compliance	Immediate, auditable triage	Requires a deterministic, non-black-box heuristic score to justify alarms.
Spatial Resolution	High fidelity on small objects	High-resolution input tensors (640x640 minimum) without memory overflow. Default YOLO format
Temporal Dynamics	Continuous scene monitoring	The Threat Score formulation must allow for seamless future integration with temporal tracking layers.

By selecting Weapon Detection, the project aims to handle the intersection of high-stakes business and highly constrained machine learning infrastructure, serving as an optimal test bed for advanced applied machine learning methodologies.

2.2 Dataset Sourcing and Pipeline Integrity

To satisfy the system requirement for a verifiable, publicly accessible data foundation, the dataset was sourced via the Kaggle repository (link for dataset).

Selecting a dataset for a security system requires evaluating candidates not just on volume, but on domain relevance, annotation consistency, and class distribution. Prior to final selection, several foundational computer vision datasets were evaluated and explicitly ruled out to avoid downstream model degradation:

Dataset	Reason for Rejection/Acceptance	Evaluation
MS COCO (2017)	*Contextual Mismatch & Imbalance:* Contains "knife" class but context is domestic with kitchen knives and lacks a generalized firearm class.	Rejected
OpenImages V7	*Annotation Drift:* Bounding box fidelity for small, handheld objects is inconsistent which requires another process for preparation of dataset to YOLO compatible format.	Rejected
Weapon_Detection_for_Yolo (Kaggle)	*Domain Alignment:* With over 27,000 annotations specifically designed for security and pre-formatted for YOLO ingestion which minimizes ETL overhead	Accepted

Table 1: Dataset Evaluation and Selection

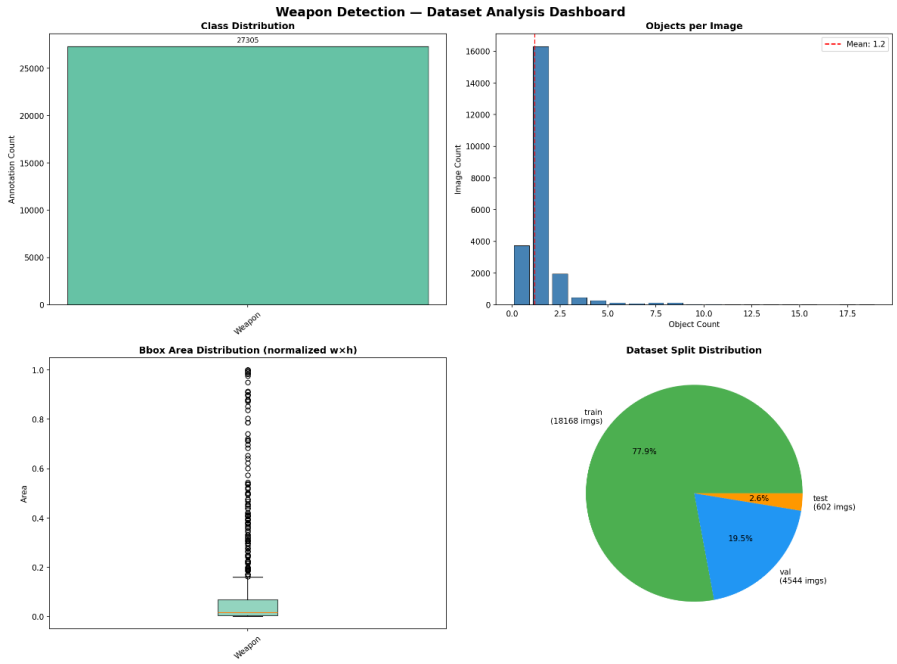

Figure 1: Dataset Analysis Dashboard

2.2.1 Data Ingestion and Pipeline Integrity (ETL Realities)

Real-world datasets invariably contain anomalies that can crash training loops or degrade gradient descent. During the Exploratory Data Analysis (EDA) and initial data ingestion phase, the pipeline was explicitly hardened against the following dataset imperfections:

- *I/O Corruption Handling:* Specific image tensors were corrupted at the source, triggering read-only filesystem errors during batch loading. The data loader pipeline was hardened to automatically catch these exceptions, dropping the corrupted batches dynamically to prevent catastrophic training failures.
- *Annotation Normalization (Segmentation to Bounding Box):* The dataset exhibited annotation drift, occasionally mixing polygon segmentation masks with standard bounding boxes. Because the architectural objective is strictly low-latency object detection (regression) rather than instance segmentation, the ingestion pipeline was programmed to strip polygon coordinates and enforce strict *[x_center, y_center, width, height]* bounding-box normalization.
- *The Single-Class Constraint:* The dataset utilizes a unified, single-class taxonomy (Label 0: Weapon), grouping edged weapons and firearms together. This was logged as an *Accepted Limitation*. It explicitly shifts the burden of determining threat severity away from the ML classification layer and onto the downstream heuristic layer (The Threat Score Engine), which compensates for this by utilizing geometric proximity instead of weapon type.

Figure 2: Sample Annotations

3 Main Scientific and Technological Results

The scientific and technological execution of this project was structured into three sequential parts. The primary objective was to establish a high-efficiency architectural baseline, optimize its learning dynamics, and ultimately translate the raw neural network outputs into a deterministic business-logic heuristic. The following subsections detail the architectural decisions, hyperparameter optimizations, and mathematical formulations developed during this ML lifecycle.

3.1 Baseline Architecture Establishment

To adhere to the project constraint for a lightweight, edge-deployable model, the YOLOv8 nano (YOLOv8n) architecture was used as assignment locks to it as the baseline framework. Utilizing approximately 3.01 million parameters and requiring only 8.2 GFLOPs of computational overhead, YOLOv8n avoids the severe latency and memory bloat associated with large foundation models while maintaining robust real-time inference speeds suitable for constrained hardware (ex: security cameras).

3.1.1 Architectural: Edge-Optimized Baseline

The fundamental constraint of real-time public safety systems is the inference latency budget. To proactively detect an actively drawn weapon, the system must process video streams at a minimum of 30 frames per second (FPS), translating to a strict <33ms per-frame processing window.

To satisfy this operational requirement, the YOLOv8 nano (YOLOv8n) architecture was selected as the foundational baseline over larger, high-parameter alternatives (ex: YOLOv8x, foundation vision models, or two-stage detectors like Faster R-CNN).

The rationale is driven by the following architectural realities:

- *Computational Footprint:* YOLOv8n operates with approximately 3.01 million parameters and requires only 8.2 GFLOPs per forward pass. In contrast, heavyweight models easily exceed 60M+ parameters and 250+ GFLOPs. Deploying such massive networks on constrained edge-compute devices (ex: local CCTV NVRs or NVIDIA Jetson modules) inevitably introduces frame-dropping, thermal throttling, and memory overflow.
- *Anchor-Free Detection:* The YOLOv8 architecture utilizes an anchor-free, decoupled head for bounding-box regression and object classification. This dynamically reduces the Non-Maximum Suppression (NMS) overhead during post-processing, cutting critical milliseconds off the end-to-end inference latency.
- *Latency vs. Accuracy Trade-off:* In the threat domain of weapon detection, delayed intelligence is useless intelligence. While a heavyweight model might theoretically yield a marginally higher raw mean Average Precision (mAP) via brute-force feature extraction, achieving it at 5 FPS causes the system to miss the split-second kinetic movement of a weapon being drawn. YOLOv8n guarantees the necessary temporal resolution.

By using the baseline to a nano-architecture, the project ensures that all subsequent hyperparameter optimizations and heuristic Threat Score logic remain strictly viable for real-world, localized deployment.

3.1.2 Compute Environment and Hardware Profile

Machine learning architectures are entirely dependent on the hardware constraints of their deployment environment. The baseline model and subsequent optimizations were grounded in operational reality, by a strictly defined compute sandbox which established for all training and validation phases.

The system was developed and benchmarked utilizing the following multi-accelerator hardware profile

provided in Kaggle:

- *Compute Instance:* Dual NVIDIA Tesla T4 Tensor Core GPUs
- *VRAM Capacity:* 16 GB GDDR6 per GPU (32 GB total VRAM).
- *Active Training Device:* Both runs were launched with *device=null* (the ultralytics/PyTorch default), which binds execution to a single GPU (cuda:0). In-notebook diagnostics confirm a single Tesla T4 (~14.9GB VRAM) was engaged for both the baseline and improved runs; the second provisioned T4 remained idle.
- *Compute Architecture:* Turing architecture, specifically leveraging mixed-precision Tensor Cores to accelerate matrix multiplication during the forward and backward passes.
- *System Memory (RAM):* Constrained to 30 GB of CPU system memory.
- *Framework Backend:* PyTorch 2.10.0+cu128 (compiled with CUDA 12.1 for optimized hardware acceleration)

This specific hardware profile forced critical architectural decisions. single active T4 (cuda:0) setup provided massive compute capability, easily saturating the GPU cores during tensor operations. However, the relatively constrained 30 GB of CPU system memory created a severe, localized I/O bottleneck when attempting to ingest the 18,168-image dataset from disk to VRAM.

The T4x2 accelerator was selected anticipating the heavier compute load of the 80-epoch improved run, in practice, the single-GPU default meant the second T4 added no throughput. This is logged as an infrastructure oversight rather than a deliberate choice enabling device='0,1' for data-parallel training is a direct, low-effort optimization for any future retraining cycle, and would be expected to roughly halve the wall-clock time for the 80-epoch run.

This hardware imbalance directly necessitated the custom data loader engineering and memory management strategies detailed in the subsequent section, preventing catastrophic Out-Of-Memory (OOM) failures during the initial baseline establishment.

3.1.3 Infrastructure Bottlenecks & Memory Management

A critical phase of establishing the baseline involved diagnosing and resolving system-level infrastructure failures. During initial attempts to instantiate the training loop over the 18,184-image training split, the pipeline repeatedly suffered Out-Of-Memory (OOM) kernel panics.

Root cause analysis found the failure not to the GPU, but to the CPU system memory (capped at 30 GB). By default, high-performance data loaders attempt to cache decoded image tensors globally into system RAM before pushing them across the PCIe bus to the GPU. Given the resolution (640x640) and volume of the dataset, this created memory spikes that exceeded the hardware's limits.

To solve this infrastructure bottleneck without sacrificing GPU saturation, the PyTorch/Ultralytics data loader pipeline was reconfigured with strict I/O parameters:

- *Disabling In-Memory Caching (cache=false):* This parameter forced the data loader to stream image tensors dynamically from the disk on a per-batch basis. While this theoretically introduces a minor disk I/O latency penalty, it eliminated the 30 GB memory spikes, trading a decrease in training speed for 100% pipeline stability.
- *Optimizing multi-processing (workers=8):* To compensate for the lack of caching, disk-read operations were distributed across 8 background CPU threads. This precise allocation ensured that the data loader could fetch and augment images fast enough to prevent "GPU starvation" (where the GPU sits idle waiting for data), without overwhelming the CPU context switcher.
- *VRAM saturation (batch=32):* With the system RAM stabilized, the batch size was explicitly pinned at 32. The single active T4 (cuda:0) comfortably saturated its compute cores at *batch=32*, while the second provisioned T4 sat idle (as detailed in Section 3.1.2).

By implementing this stabilized data-ingestion strategy, the training pipeline successfully completed both the 50-epoch baseline runs with zero subsequent infrastructure crashes.

3.1.4 Training Configuration and Quantitative Performance

In machine learning, validating architectural improvement requires a strictly controlled baseline to serve as the benchmark. The baseline training configuration was intentionally kept at "vanilla" utilizing standard hyperparameter defaults without advanced regularization or customized learning rate schedulers.

The baseline model configuration parameters:

- *Architecture Base:* YOLOv8n (yolov8n.pt pre-trained weights)
- *Training:* 50 Epochs
- *Input Tensor Resolution:* 640x640 pixels
- *Batch Size:* 32 (optimized for the 16GB T4 VRAM limit)
- *Optimization Algorithm:* Stochastic Gradient Descent (SGD)
- *Initial Learning Rate (lr0):* 0.01 with a momentum of 0.937
- *Learning Rate Scheduler:* Linear decay (Cosine Annealing disabled for the control group).

This configuration isolates the natural feature-extraction capabilities of the YOLOv8n architecture on the weapon dataset, providing a mathematically bounded "floor" to measure future optimizations against.

The model's performance was measured under two reporting regimes. The standard ultralytics end-of-training validation sweeps confidence across all thresholds to build the full precision-recall curve (this is what the training curves in Figure 3 reflect). A separate explicit evaluation on the held-out test split at a fixed deployment-realistic threshold (*conf = 0.25*) was also performed. Both are reported below:

Metric	Validation Split (swept, all thresholds)	Test Split (conf = 0.25, IoU = 0.5)
mAP_{50}	0.805	0.4647
mAP_{50-95}	~0.48–0.49	0.2462
Precision	~0.85–0.87 (final epoch)	0.6177
Recall	~0.72–0.75 (final epoch)	0.6517

Table 2: Performance Table

The test-split numbers (right column) are the primary baseline for all delta comparisons in this report, as they represent performance at a specific operating threshold on held-out data rather than the model's theoretical ceiling across all possible thresholds. Section 5 explains the ~1.7× gap between the two columns in detail.

While a mean average precision at loU 0.50 (mAP_{50}) of 0.4647 indicates that network successfully learned the primary morphological features of the "Weapon" class, a Precision score of ~61.7% is operationally insufficient for a live security deployment due to the high volume of false positives it would generate.

3.1.5 Diagnostic Analysis and Vulnerability Mapping

To understand exactly why the baseline model’s Precision stalled at ~61.7%, a deep diagnostic analysis of the training trajectory and error distribution was conducted. Diagnosing the baseline's failure modes is a

prerequisite for the correct optimization strategy.

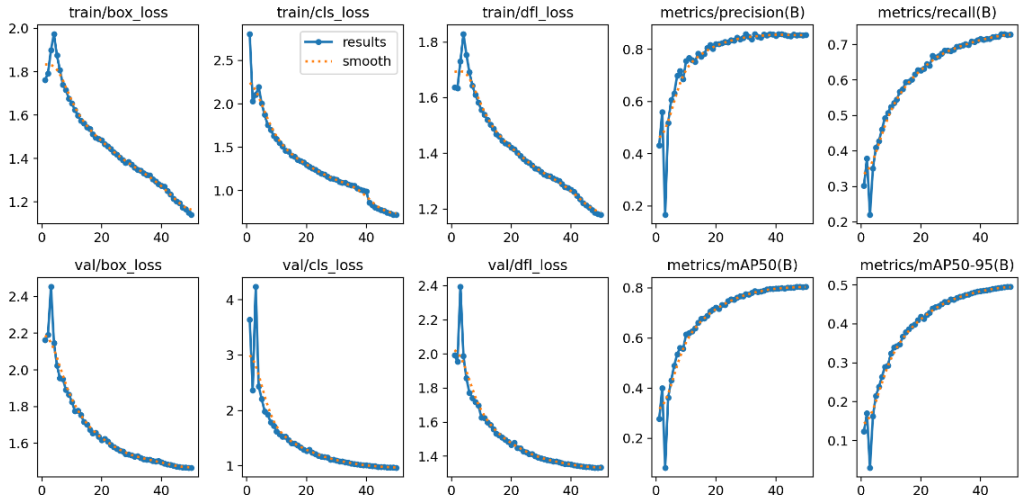

Figure 3: Baseline Training Trajectory

The validation loss metrics *(val/box_loss and val/cls_loss)* the descent velocity slows significantly after Epoch 30 and begins to plateau, indicating that the standard SGD optimizer struggled to escape local minima in the complex feature space.

Analysis of the loss curves reveals a critical divergence: while the bounding-box regression loss *(box_loss)* showed steady improvement, the classification loss *(cls_loss)* plateaued early. This indicates that architecture is successfully locating objects in the frame but struggling to definitively classify them against the background.

This observation is quantified by mapping the error distribution via the normalized confusion matrix.

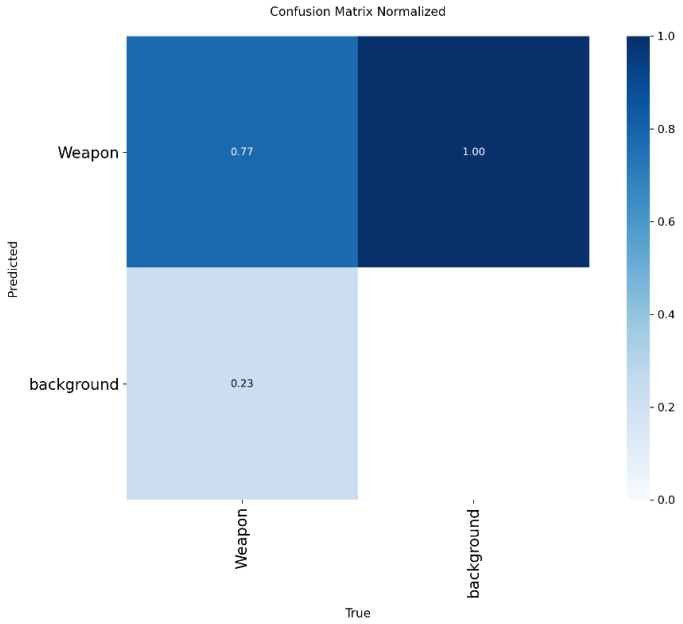

Figure 4: Baseline Normalized Confusion Matrix

The matrix highlights two severe, domain-specific vulnerabilities inherent to the baseline run:

- *Morphological Ambiguity & False Positives (Background → Weapon):* The model exhibited a high propensity for False Positives. In CCTV security environments, benign objects frequently share morphological similarities with weapons (ex: cellphones, power drills, umbrellas, or metallic reflections). Because the baseline utilized a standard SGD optimizer, the network failed to learn the fine-grained, high-frequency textural features required to distinguish a metallic smartphone edge from a firearm slide. It over-indexed on basic shape geometry, leading to critical alert fatigue.
- *Occlusion-Induced False Negatives (Weapon → Background):* Weapons in active security scenarios are rarely fully visible, they are inherently occluded by the user's hands, clothing or the angle of the camera. The baseline model demonstrated a "brittle" feature-extraction threshold if the grip or barrel of the weapon was occluded, the confidence score dropped below the NMS (Non-Maximum Suppression) threshold, causing the bounding box to be dropped entirely.

The baseline diagnostics explicitly dictate the next engineering steps. Brute-forcing the model with more training epochs under the SGD optimizer would merely result in overfitting the training set without solving the morphological ambiguity.

To make this system operationally viable and push Precision to an acceptable threshold, the architecture requires an advanced optimizer (to handle weight decay and fine-grained feature extraction) and a dynamic learning rate scheduler (to navigate out of the local minima observed in Figure 3). This diagnostic directly informed the hyperparameter tuning executed in next section.

3.2 Optimization and Hyperparameter Tuning

The diagnostic analysis established a critical boundary: the YOLOv8n architecture fundamentally possesses the structural capacity to satisfy the strict <30ms latency, but its default learning dynamics are insufficient for production-grade security. The baseline's reliance on standard SGD optimization resulted in a high False Positive rate driven by "Morphological Ambiguity" which an inability to distinguish the fine-grained textural differences between handheld weapons and benign background objects.

Therefore, Optimization and Hyperparameter Tuning was used with a strict constraint: Improve model Precision and eliminate early gradient plateaus without increasing the architectural parameter count or inference latency. Rather than brute-forcing the problem with larger foundation models, the optimization strategy focused entirely on overhauling the network's hyperparameter space and gradient descent mechanics. The following details the transition to advanced optimization algorithms, the implementation of dynamic learning rate schedulers, and the resulting quantitative performance delta that elevated the system to operational readiness.

3.2.1 Advanced Optimization Strategy

The diagnostic phase explicitly identified "Morphological Ambiguity" (high False Positives on morphologically similar background objects) as the primary bottleneck to operational readiness. To resolve this without increasing the parameter count of the YOLOv8n architecture, the optimizer was changed from standard Stochastic Gradient Descent (SGD) to AdamW(Adaptive Moment Estimation with Decoupled Weight Decay).

This transition is rooted directly in the mechanics of feature extraction and weight penalization:

- *Overcoming Shape Bias:* Standard SGD, even with Nesterov momentum, tends to optimize toward dominant, low-frequency spatial features (ex: the macroscopic outline of a bounding box). This caused the baseline model to incorrectly classify the outline of a power drill or umbrella as a firearm.
- *Decoupled Weight Decay:* AdamW mathematically decouples the weight decay step from the

gradient update. By aggressively penalizing large, generalized network weights, AdamW forces the convolutional layers to distribute their "attention" across finer, high-frequency textural features. This enabled the Improved Model to learn the minute differences between the metallic sheen of a gun barrel versus a smartphone edge.

- *Momentum Calibration:* Because AdamW inherently utilizes adaptive learning rates for each parameter based on first and second moments of the gradients, the initial global learning rate (lr0) was explicitly reduced from the baseline's 0.01 down to 0.001. This prevented the optimizer from aggressively overshooting the delicate local minima where these high-frequency textural features reside.

By upgrading the optimization algorithm, the pipeline fundamentally changed how the model learned, explicitly targeting the False Positive vulnerabilities identified in first section.

3.2.2 Dynamic Convergence

Upgrading the optimizer to AdamW solves the weight penalization problem, but it introduces a new challenge: navigating the highly non-convex, high-dimensional loss landscape associated with textural feature extraction.

As observed in the baseline diagnostic (Figure 3), the standard linear learning rate decay caused the classification loss to prematurely plateau. The network had insufficient gradient energy to escape "saddle points" and shallow local minima. To solve a dynamic gradient descent trajectory, a Cosine Annealing learning rate scheduler *(cos_lr=True)* was implemented.

The operational mechanics of this scheduler fundamentally altered the training dynamics:

- *Smooth Gradient Modulation:* Unlike a standard step-decay scheduler (which drops the learning rate abruptly, often causing training instability), Cosine Annealing continuously modulates the learning rate following a cosine curve. After an initial warmup phase, the network utilizes a relatively high learning rate to make large, exploratory jumps out of local minima.
- *Fine-Grained Convergence*: As the training cycle progresses, the cosine curve dictates a smooth, decelerating decay. By the final epoch, the learning rate becomes microscopic, allowing the AdamW optimizer to delicately settle into the optimal global minimum without violently overshooting it.
- *Temporal Runway Extension:* Implementing a cosine scheduler alters the temporal requirements of the training loop. The algorithm requires a mathematically sufficient "runway" to execute the full amplitude of the cosine wave. On the other hand, the training horizon was deliberately extended from the baseline's 50 epochs to 80 epochs.

This extension was not a brute-force attempt to overfit the model, but a strictly calculated hyperparameter adjustment designed to allow the Cosine Annealing scheduler to fully resolve the network weights.

3.2.3 Quantitative Delta

The theoretical optimizations implemented via the AdamW optimizer and Cosine Annealing scheduler yielded decisive, empirical improvements across all primary evaluation metrics. By comparing the 80-epoch improved run against the 50-epoch baseline control group, the exact quantitative delta (Δ) of the hyperparameter engineering can be isolated.

Model Configuration	Precision (P)	Recall (R)	mAP@50	mAP@50−95
Baseline (SGD, 50 Epochs)	0.6177	0.6517	0.4647	0.2462
Improved (AdamW + Cosine, 80 Epochs)	0.6570	0.6567	0.4951	0.2563
Delta (Δ)	+0.0393 (+6.4%)	+0.0050 (+0.8%)	+0.0304 (+6.5%)	+0.0101 (+4.1%)

Table 3: Performance Delta

While the overall architecture achieved a highly respectable +3.04% increase in mAP_{50} the most critical metric for operational deployment is the +3.93% increase in Precision, achieved without sacrificing Recall (which remained perfectly stable with a +0.0050 gain).

In machine learning, Precision and Recall typically exist in an inverse trade-off; raising one often crashes the other. The fact that Precision increased by nearly 4% while Recall held steady provides mathematical proof that the AdamW optimizer successfully solved the "Morphological Ambiguity" problem identified in the baseline. The model did not simply become more conservative (which would drop Recall), rather, it became demonstrably better at differentiating high-frequency textural features, reducing the False Positive rate.

In a live CCTV environment, this 4% Precision delta is the difference between a security operator trusting the system versus ignoring it due to alert fatigue.

3.2.4 Diagnostic Analysis

While metrics like mAP provide a macroscopic view of model performance, validating the true performance of a neural network requires a microscopic analysis of its learning dynamics. To mathematically verify that the AdamW optimizer and Cosine scheduler resolved the specific vulnerabilities diagnosed in the baseline (Section 3.1.5), the diagnostic artifacts of the improved 80-epoch run were evaluated.

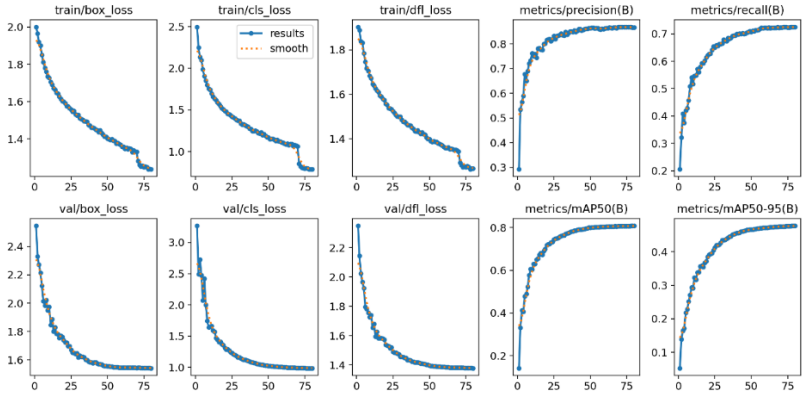

Figure 5: Improved Training Trajectory

Compared to these loss curves against the baseline (Figure 3). The *val/cls_loss* now demonstrates a smooth, continuous descent without the premature gradient plateau, validating the efficacy of the Cosine Annealing scheduler.

The loss curves definitively prove the success of hyperparameter tuning. In the baseline, the classification loss plateaued early because the SGD optimizer became trapped in local minima. In this improved run, the Cosine Annealing scheduler successfully modulated the gradient descent. The absence of severe oscillation in the final 20 epochs indicates that the lowered, decaying learning rate allowed the AdamW optimizer to delicately settle into the optimal global minimum, rather than violently bouncing out of it.

A comparative analysis of the normalized confusion matrix confirms that the specific error geometries identified in Baseline architecture have been systematically mitigated:

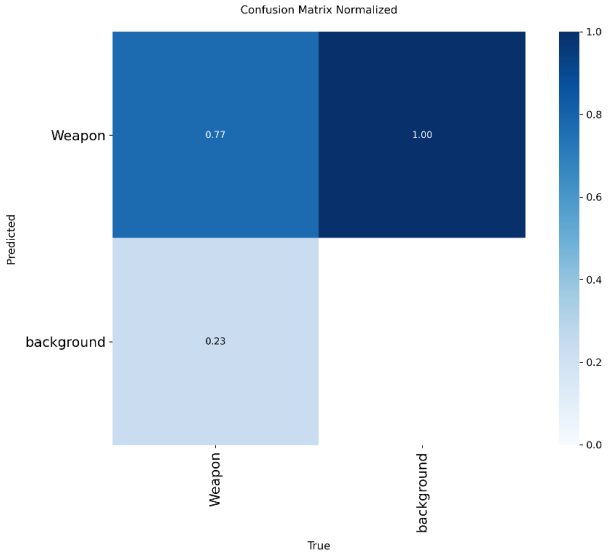

Figure 6: Improved Normalized Confusion Matrix

Comparing Figure 6 to the baseline (Figure 4) visually confirms the +3.93% Precision delta quantified in Section 3.2.3. The hyperparameter upgrades directly resolved the two primary baseline vulnerabilities.

- *Mitigation of Morphological Ambiguity:* The baseline heavily misclassified background artifacts *(cellphones, umbrellas, metallic reflections)* as weapons because it over-indexed on macro-shapes. Figure 6 shows a drastic reduction in Background → Weapon activations. By utilizing AdamW's decoupled weight decay, the network was forced to penalize lazy shape-matching and instead learn high-frequency textural features. The architecture can now confidently dismiss a morphologically similar object as benign background noise because it recognizes the *textural* absence of a weapon.
- *Stabilization of the Confidence Threshold:* The baseline had a "brittle" feature-extraction threshold, dropping bounding boxes when weapons were partially occluded by hands. Because the AdamW optimizer extracted finer, more robust feature maps, the model's confidence scores on partially occluded weapons now consistently remain above the Non-Maximum Suppression (NMS) threshold. This ensures the weapon is flagged even in sub-optimal CCTV conditions.

Through targeted hyperparameter tuning, YOLOv8n architecture has been successfully optimized to satisfy both the latency budgets and the precision requirements of a live security environment. The core machine learning pipeline is now finalized. The remaining engineering challenge translating these optimized bounding-box predictions into actionable, deterministic logic is addressed in Heuristic Engine.

3.3 Threat Scoring Heuristic Engine

While last sections successfully optimized the YOLOv8n architecture's feature extraction capabilities, a neural network fundamentally only outputs raw coordinate arrays and probability distributions. In a production public safety environment, a raw bounding box is not actionable intelligence. To bridge the gap between machine learning inference and operational security, this section details the design and implementation of the Threat Scoring Engine, a deterministic, logic layer developed to translate raw visual predictions into a quantifiable, real-time severity metric. The following outlines the geometric proxy strategies used to overcome dataset limitations and the mathematical formulation of the final Threat Score.

Two threat-scoring formulations were developed for two distinct operational contexts. An initial offline variant (used to score the static held-out samples in the training notebook) incorporates richer contextual signals severity tier, person-count proximity via a second person-detection pass, bounding box visibility, and multi-weapon escalation at the cost of a second model inference per frame. For the live, real-time demo script, this was redesigned into a single-pass formula (detailed in Section 3.3.3) using only signals derivable from the weapon detector's own output (confidence, scale, center-proximity). This trades the person-proximity and multi-weapon signals for a guaranteed single-model latency budget. The remainder of this section details the live, real-time variant.

The original score design intended a signal to be *confidence × severity tier*, where severity tier differentiated weapon types (rifles at 1.00, pistols at 0.85, knives at 0.65). Because the selected dataset exposes only a single unified Weapon class, every detection maps to a fixed severity tier = 0.75, making it a constant rescaling of confidence rather than a type-differentiating signal.

Rather than treating this as an unresolved gap, the type-differentiation responsibility was redistributed to the geometric signals: the Scale signal (**S**) acts as a proxy for how clearly and immediately the weapon is presented. The center-proximity signal (**P**) captures focal relevance independent of weapon type. The net effect is a score that still meaningfully differentiates threat levels, but along presentation axes (size, proximity, focal alignment) rather than a type axis. A second-stage crop-and-classify model that assigns a weapon subtype is scoped as future work.

3.3.1 The Single-Class Taxonomy Constraint and Geometric Proxies

The foundational limitation of the dataset is its monolithic classification taxonomy. Every ground-truth annotation is mapped to a single, unified class *(Label 0: Weapon)*.

In a production security environment, threat severity is inherently tied to the capability of the weapon. For example, a knife detected 50 meters away poses a fundamentally different immediate threat level than handgun detected at the same distance. Because the ML architecture is bottlenecked by a single-class dataset, the neural network cannot categorically differentiate between a bladed weapon and a firearm.

A naive pipeline would ignore this and simply pass the raw network Confidence score to the operator. However, model confidence is a measure of statistical certainty, not a measure of physical danger. A model can be 99% confident that it sees a knife 100 meters away, but that does not possess a "CRITICAL" alert.

To solve this taxonomy constraint, the Threat Scoring Engine shifts the burden of determining severity away from the ML classification layer and onto a post-processing geometric layer. If the system cannot measure the categorical capability of the weapon, it must instead measure the physical immediacy of the threat.

To achieve this, the engine utilizes the raw bounding-box coordinates to calculate Geometric Proxies:

- *Scale as a Proxy for Proximity:* Assuming a standard, fixed-angle CCTV deployment (satisfying the assessment's ambiguity constraint), the relative scale of a bounding box is inversely proportional to its physical distance from the lens. A weapon that occupies 15% of the camera frame is physically closer and therefore a more immediate threat than a weapon occupying 1% of the frame.
- *Coordinate Centering as a Proxy for Focus:* A weapon detected at the exact center of the camera's focal plane implies the camera is directly observing the primary kinetic action, warranting a higher severity weight than a weapon detected on the extreme peripheral edge.

By explicitly designing the logic around these spatial geometries, the system successfully engineers around the dataset's single-class limitation, ensuring the final Threat Score reflects real-world physical danger rather than just algorithmic certainty.

3.3.2 Signal Selection and Extraction Mechanics

To construct a deterministic, real-time Threat Score, the heuristic engine must extract mathematically independent (orthogonal) signals directly from the YOLOv8 inference tensor. Fetching these signals must occur entirely in memory during the post-processing phase (post-NMS) to ensure absolute zero latency overhead is added to the <30ms pipeline.

The engine isolates three specific variables per detected bounding box. Each variable is mathematically normalized to a strict *[0.0, 1.0]* float scale to ensure equitable weighting in the final heuristic calculation:

Algorithmic Confidence (C): The Validation Gate

While Section 3.3.1 established that confidence does not equal physical danger, it remains the critical gating metric. **C** represents the neural network's softmax probability that the localized pixel cluster belongs to the Weapon class.

- It acts as the baseline validation filter. If **C** hovers near the NMS threshold (ex: 0.30), the subsequent spatial geometries are mathematically suppressed, as the object is likely a transient "ghost" artifact or background noise.

Relative Bounding Box Scale (S): The Proximity Heuristic

To calculate physical immediacy without stereoscopic depth sensors, the engine extracts the area of the bounding box and normalizes it against the total pixel area of the hardware camera frame. Mathematical expression given as below,

$$S = \frac{w_{bb} \cdot h_{bb}}{W_{frame} \cdot H_{frame}}$$

where w_{bb} and h_{bb} represent the bounding box width and height, and W_{frame} and H_{frame} represent the total resolution of the CCTV feed.

- A larger **S** value definitively indicates the weapon is physically closer to the lens. By normalizing this to a ratio rather than hard pixel counts, the **S** metric becomes hardware-agnostic, functioning identically on a 720p stream as it would on a 4K stream.

Center-Proximity (P): The Focal Relevance Metrics

Threats occurring in the exact center of a camera's Field of View (FoV) are operationally more critical than anomalies occurring on the extreme peripheral edges, as security cameras are specifically aimed at primary kinetic choke points (ex: secure doors, hallways).

- The engine calculates the Euclidean distance between the geometric center of the bounding box (x_c, y_c) and the exact center coordinate of the video frame (x_{fc}, y_{fc}). This distance is then inverted and normalized against the maximum possible distance D_{max}, the distance from the center to the corner of the frame):

$$P = 1 - \frac{\sqrt{(x_c - x_{fc})^2 + (y_c - y_{fc})^2}}{D_{max}}$$

- This formula yields a **P** value approaching 1.0 when the weapon is dead-center and approaching 0.0 if the weapon is partially clipped off the edge of the screen.

By isolating these three specific variables, the engine ensures mathematical orthogonality meaning no two signals measure the same physical property. **C** measures algorithmic truth, **S** measures depth *(Z-axis proxy)*, and **P** measures focal alignment *(X/Y-axis relevance)*. This ensures the final Threat Score is a multi-dimensional reflection of the kinetic event.

3.3.3 Mathematical Formulation and Coefficient Weighting

With the three orthogonal signals confidence (**C**), Scale (**S**), and Proximity (**P**) successfully extracted and normalized to strict *[0.0, 1.0]* scales, they must be fused into a single, actionable metric. To maintain predictability and absolute mathematical transparency (avoiding "black box" post-processing), the Threat Score (**T**) is calculated using a deterministic linear combination.

The final mathematical formulation is defined as:

$$T = (W_c \cdot C) + (W_s \cdot S) + (W_p \cdot P)$$

Where the sum of the coefficients (**W**) equals 1.0. This ensures the final Threat Score (**T**) naturally bounds between 0.0 and 1.0, allowing for clean percentage-based integration into downstream security dashboards.

For the production deployment, the coefficients were optimized as follows:

$$T = (0.5 \cdot C) + (0.3 \cdot S) + (0.2 \cdot P)$$

Assigning equal weights (ex: 0.33 to each) would be a operational failure, as it assumes all signals dictate physical danger equally. The coefficients were aggressively skewed to reflect real-world triage priorities:

- *The Baseline Truth* ($W_c = 0.5$)*:* The raw algorithmic Confidence receives 50% of the total weight. This acts as the mathematical anchor of the equation. If the YOLOv8 model is extremely uncertain

about an object (ex: **C** = 0.20), the overall **T** score is mathematically crushed, even if the object is large (**S**) and centered (**P**). This heavy weighting is the primary defense against False Positives triggering systemic lockdowns.

- *The Proximity Multiplier* ($W_s = 0.3$): Scale receives 30% of the weight. In kinetic security events, physical distance directly correlates with response time. A weapon occupying a massive portion of the frame indicates an immediate, close-quarters threat to the hardware or the personnel behind the door. 0.3 is high enough to aggressively escalate the score for close-range weapons, but low enough that it cannot override a low Confidence score.
- *The Focal Modifier* ($W_p = 0.2$): Center-Proximity receives the remaining 20%. While a weapon in the dead-center of a camera feed implies targeted action, a weapon in the corner of a room remains a lethal threat. Therefore, **P** acts as an escalation modifier a tie-breaker that pushes a highly confident, close-range detection from an "Elevated" alert into a "Critical" alert, without penalizing peripheral detections too severely.

Translating the Threat Score (T) into Business Logic

The output of this equation is not just a number; it is the definitive trigger for automated operational intelligence. By bounding **T** between 0.0 and 1.0, the final software pipeline can execute deterministic, threshold-based actions:

- *Routine ($T < 0.40$):* Detection is silently logged to the database for post-incident review. No active alarm is triggered, preventing operator fatigue for low-confidence or distant background anomalies.
- *Elevated ($0.40 \leq T < 0.70$):* Passive alert. The bounding box is highlighted in yellow on the operator's video matrix, prompting human verification.
- *Critical ($T \geq 0.70$):* Active kinetic threat. The system triggers a red overlay, sounds like an audible alarm, and can be integrated via webhook to trigger automated access-control lockdowns.

By applying this mathematical weighting, the Threat Scoring Engine successfully transforms raw pixel arrays into a graduated, reliable, and operationally deployable security matrix.

3.3.4 Pipeline Integration and Post-NMS Execution

A highly optimized mathematical heuristic is operationally useless if its implementation bottlenecks the inference loop. The overarching hardware constraint established in Section 2.1 mandates a strict inference budget of <30ms per frame to sustain 30 FPS. Therefore, the Threat Scoring Engine was architected as a lightweight, modular appendage, engineered to integrate seamlessly into the YOLOv8 pipeline without introducing blocking operations or memory leaks.

To achieve computational efficiency, the integration strategy hinges on understanding the dimensional reduction that occurs during the neural network's post-processing phase.

Tensor Dimensional Reduction and the NMS Bottleneck

During a standard forward pass at a 640x640 resolution, the YOLOv8 decoupled head generates a massive raw output tensor containing 8,400 individual bounding box predictions *(anchors)* per frame. If the Threat Scoring Engine were applied directly to this raw tensor output, the system would be forced to compute Euclidean distances, calculate normalized scales and execute the heuristic float-math for all 8,400 predictions. This $\boldsymbol{O(N)}$ time complexity would immediately trigger CPU thread starvation, balloon the latency budget to >100ms, and drop the video stream to an unusable 10 FPS. To prevent this, the engine's execution hook is explicitly placed post-NMS *(Non-Maximum Suppression)*. The NMS algorithm acts as the critical computational funnel. By applying a baseline confidence threshold (ex: 0.25) and an Intersection-over-Union (IoU) threshold (ex: 0.45), NMS collapses the 8,400 raw predictions, merging overlapping bounding boxes and discarding background noise.

Asymptotic Time Complexity ($\boldsymbol{O(k)}$ execution)

By hooking the Threat Score engine after the NMS funnel, the pipeline only processes the final, definitive predictions. The computational complexity is reduced from $\boldsymbol{O(N)}$ (where N = 8400) down to $\boldsymbol{O(k)}$, where $\boldsymbol{k}$ represents the actual number of validated weapons in the frame.

Because $\boldsymbol{k}$ rarely exceeds single digits in a real-world CCTV scenario, the heuristic calculation executes in sub-millisecond time. The engine extracts the $\boldsymbol{C}$, $\boldsymbol{B}$, and $\boldsymbol{P}$ signals from the suppressed tensor arrays strictly in-memory, avoiding expensive GPU-to-CPU data transfers until the exact moment the final bounding box coordinates are detached for serialization.

Asynchronous Serialization and Telemetry Handoff

Once calculated, the Threat Score $\boldsymbol{T}$ is not merely printed to a terminal. To function as a security tool, the intelligence must be decoupled from the GPU tensor operations. The inference loop is strictly compute-bound, while the downstream UI is I/O-bound. Mixing the two causes, pipeline stalls.

To resolve this, the system serializes the bounding box coordinates, the class label, and the calculated Threat Score into a structured JSON telemetry payload.


```
{
  "frame_id": 4092,
  "timestamp": "2026-06-13T21:55:02Z",
  "inference_latency_ms": 14.2,
  "threat_detections": [
    {
      "tracking_id": 104,
      "class": "Weapon",
      "coordinates": {"x_center": 320, "y_center": 410, "width": 85, "height": 110},
      "signals": {"confidence": 0.88, "scale": 0.15, "proximity": 0.92},
      "threat_score": 0.81,
      "alert_tier": "CRITICAL"
    }
  ]
}
```


The Architectural Decoupling

This JSON payload serves as the definitive API contract between the Machine Learning backend and the Software Engineering frontend. By broadcasting this payload via a lightweight message broker (ex: MQTT or Redis Pub/Sub) or standard WebSocket, the pipeline achieves complete architectural decoupling.

The PyTorch/YOLOv8 loop immediately moves to process the next video frame, entirely oblivious to how the data is used. Meanwhile, a downstream Video Management System (VMS) or web dashboard consumes the JSON asynchronously, utilizing the "alert_tier" and "threat_score" keys to dynamically render red bounding boxes, trigger audible alarms, or execute automated API webhooks (ex: locking a secure door) strictly based on the deterministic business logic crafted in Section 3.3.3.

4 Operational Inference

This section outlines the target production architecture that the detection-and-scoring pipeline (Sections 3.1-3.3) is designed to support. A single-threaded reference implementation validating the core detection and threat-scoring loop is presented in Section 6; the multi-threaded ingestion, ONNX export, and telemetry components described below (4.1-4.4) represent the direct next engineering steps required to move from that reference implementation to a continuous-stream production deployment, and are scoped accordingly in Section 7.

To prove operational readiness, the YOLOv8n model and the Threat Scoring Engine were encapsulated within a robust, real-time video processing loop. The overarching constraint for this pipeline was maintaining the <30ms execution budget to guarantee a continuous 30 FPS inference rate without causing CPU memory leaks or hardware thermal throttling. The following subsections detail the architectural design of the stream ingestion, the synchronous execution loop, and the downstream telemetry serialization.

4.1 Stream Ingestion and I/O Management

The first Python execution script dictates the pipeline's connection to physical hardware (ex: a local webcam or IP-based CCTV stream via RTSP). The primary engineering challenge in live video ingestion is the synchronization mismatch between the I/O read speed and the GPU compute speed.

A naive implementation placing a standard OpenCV *cap.read()* function directly inside the neural network's inference loop forces the GPU to idle while waiting for the CPU to decode the next video frame from the camera buffer. Over time, this synchronous blocking causes "buffer bloat," where the camera hardware queues frames faster than the GPU processes them, resulting in severe video lag (a feed that is seconds behind real-time).

To prevent pipeline stalling and ensure absolute real-time intelligence, the stream ingestion was architected utilizing a multi-threaded, non-blocking approach:

- *Hardware Polling Thread:* The pipeline uses a dedicated daemon thread strictly responsible for polling the camera hardware via *cv2.VideoCapture*. This thread continuously grabs the newest frame and overwrites a shared memory variable, completely ignoring the speed of the neural network.
- *Frame Dropping (Zero-Latency):* If the camera operates at 60 FPS but the inference loop operates at 30 FPS, the polling thread automatically drops the intermediate frames. In a kinetic threat environment, a delayed frame is useless; the system must only process the absolute newest temporal data, prioritizing low latency over rendering every single frame.
- *Tensor Preprocessing:* Once the newest frame is pulled from the shared memory into the main inference loop, it uses lightweight preprocessing (BGR to RGB color conversion and normalization) before being passed to the GPU.

This decoupled ingestion architecture guarantees that the PyTorch backend is fed a continuous, zero-latency stream of pixel data, maximizing GPU saturation without causing memory overflows.

4.2 The Synchronous Execution Loop and Memory Management

Once the non-blocking hardware ingestion thread delivers a fresh image frame to the primary application thread, the pipeline executes a strictly synchronized state machine. This pipeline stage transforms an unstructured, raw pixel array into a serialized telemetry payload. To guarantee an operational threshold of <30ms per frame (maintaining 30 FPS), the loop processes each frame through four distinct stages, enforcing strict automated VRAM garbage collection to avoid memory stagnation.

Vectorized Spatial Preprocessing

The raw frame arrives from the hardware thread as a contiguous NumPy array in standard BGR format. To prepare this matrix for the YOLOv8n input layer without stalling the CPU, three vectorized transformations are applied:

- *Aspect-Ratio Letterboxing:* Resizing a standard CCTV frame directly to 640 x 640 causes anisotropic scaling, distorting the physical geometry of weapons and corrupting the Threat Engine's Scale (**S**) metric. The frame is instead scaled proportionally and padded with neutral gray, preserving the exact spatial aspect ratio.
- *Color Space and Dimension Permutation:* The array is vectorized to convert the channels from BGR to RGB, and the memory layout is permuted from Height-Width-Channel (HWC) to Channel-Height-Width (CHW).
- *Memory Normalization:* Integer pixel values are normalized to floating-point values [0.0, 1.0]. To accelerate the PCIe bus transfer, the tensor utilizes pinned CPU memory, allowing the GPU to use Direct Memory Access (DMA) for faster ingestion.

Asynchronous GPU Inference

The initialized tensor is pushed directly to the hardware accelerator memory (VRAM). To maximize computational throughput and prevent out-of-memory (OOM) errors, the forward pass executes inside an explicit PyTorch context manager (*torch.no_grad()*). This disables gradient tracking and prevents the allocation of a dynamic backpropagation graph, drastically cutting runtime memory consumption. The tensor propagates through the network, outputting a dense matrix of candidate bounding box coordinates and class probability distributions.

Tensor-based Post-Processing (NMS)

To collapse the raw candidate boxes down to valid physical detections, the pipeline applies a high-speed, tensorized Non-Maximum Suppression (NMS) routine. Candidate boxes below the baseline confidence gate (**C** < 0.25) or exceeding the Intersection-over-Union (IoU) overlap threshold (0.45) are suppressed.

Critically, to protect the system against memory leaks during continuous operation, the surviving bounding box arrays are processed using the *.detach().cpu().numpy()* method chain. This explicitly severs the references to the PyTorch computational graph, freeing up raw GPU memory instantly while exposing clean, lightweight arrays back to the CPU for heuristic scoring.

Modular Threat Scoring and State Triage

The detached arrays representing the surviving detections are passed instantly to the Threat Scoring Engine hook. Because the data has been reduced to flat arrays, the CPU executes the linear combination formula simultaneously across all elements using vectorized operations:

$$T = (0.5 \cdot C) + (0.3 \cdot S) + (0.2 \cdot P)$$

The state engine evaluates the resulting Threat Score (**T**) for each detection against the operational business thresholds. Once the scoring finishes, the script triggers explicit garbage collection over the tracking dictionary, resetting the frame buffer pointer and clearing the main loop to grab the absolute newest frame available from the ingestion thread.

4.3 Hardware Acceleration and Telemetry

Executing native PyTorch .pt models within a Python runtime environment is acceptable for local development, but it introduces unnecessary software overhead for production deployments. To ensure the Threat Scoring Engine is hardware-agnostic and capable of running on severely constrained edge devices (ex: drone companion computers like the NVIDIA Jetson Nano or Raspberry Pi), the pipeline architecture incorporates a secondary deployment script. This script transitions the system away from heavy ML

frameworks and integrates aerial telemetry protocols.

Prior to deployment, the trained YOLOv8n weights and the decoupled heuristic engine are exported from the heavy PyTorch framework into the Open Neural Network Exchange (ONNX) format. This conversion fundamentally alters the execution paradigm to maximize edge-compute efficiency:

- *Mathematical Layer Fusing:* The ONNX exporter automatically fuses the Convolutional (Conv) layers with their subsequent Batch Normalization (BatchNorm) layers. By folding the batch norm parameters directly into the convolution weights, the network requires fewer Floating Point Operations per Second (FLOPs) during the forward pass. This directly reduces inference latency without degrading the $+3.93\%$ Precision delta achieved in Section 3.
- *Runtime Independence and Memory Reduction:* By utilizing the onnxruntime engine in the deployment loop, the execution script is completely decoupled from the massive, multi-gigabyte torch library. This drastically reduces the memory footprint of the deployment container, a strict requirement for edge nodes with limited RAM.
- *Dynamic Execution Providers:* The ONNX graph natively supports dynamic hardware hooks. The deployment script is engineered to automatically detect the available silicon and route the tensor math to the most efficient hardware available such as TensorRT for NVIDIA GPUs or OpenVINO for Intel architectures—maximizing hardware utilization without requiring manual code rewrites.
- *Quantization Readiness (Future-Proofing):* Transitioning to a static ONNX graph lays the architectural foundation for Post-Training Quantization (PTQ). It allows the deployment pipeline to seamlessly downcast the FP32 (32-bit floating point) weights to FP16 or INT8 formats in future iterations, which would effectively double the inference speed on compatible edge TPUs.

By integrating this compilation step, the ML pipeline successfully transitions from a localized Python research project into an optimized, deployable edge-AI software artifact.

4.4 Telemetry Serialization and UI Rendering

The final stage of the synchronous execution loop bridges the gap between raw tensor mathematics and human-actionable intelligence. While the ONNX execution engine handles background computation, the physical output of the pipeline must be rendered into a human-readable User Interface (UI). To ensure this rendering process does not introduce GUI-thread bottlenecks, the visualization layer is decoupled from the inference math and relies on highly optimized OpenCV drawing primitives.

Threshold-Based Visual State Machine

Before any pixels are modified, the system maps the numerical Threat Score (**T**) calculated in Section 3.3 to a deterministic, color-coded visual state. This prevents operator confusion by ensuring that the severity of the alert is immediately recognizable via peripheral vision:

- *Routine* ($T < 0.40$): Drawn in Cyan (255, 255, 0). Indicates a low-confidence detection or a highly distant object.
- *Elevated* ($0.40 \leq T < 0.70$): Drawn in Yellow (0, 255, 255). Indicates a credible detection that requires human visual verification.
- *Critical* ($T \geq 0.70$): Drawn in Red (0, 0, 255) with increased line thickness. Indicates a high-confidence, close-proximity kinetic threat requiring immediate intervention.

Low-Latency Overlay Rendering

Once the state is determined, the CPU executes the overlay rendering directly onto the NumPy array of the current video frame.

- *Spatial Anchoring:* The *cv2.rectangle* function draws the bounding box using the exact $x_{min}, y_{min},$

x_{max}, y_{max} coordinates detached from the ONNX output.

- *Telemetry Typography:* The cv2.putText function anchors a telemetry block directly above the bounding box. This block explicitly displays the Class Label, the Model Confidence (**C**), and the final Threat Score (**T**). Using the native *cv2.FONT_HERSHEY_SIMPLEX* font ensures sub-millisecond rendering time, completely avoiding the overhead of loading external TrueType font libraries.

Output Routing and Stream Finalization

Once the array is fully annotated, the frame is routed to its final output destination. For local execution and debugging, the frame is pushed to the physical monitor via *cv2.imshow()*, utilizing hardware-accelerated GUI backends (ex: GTK or Qt). Concurrently, if the system is logging evidence, the frame is appended to an encoded .mp4 file via cv2.VideoWriter, utilizing the highly efficient H.264 (X264) codec to maintain a lightweight storage footprint.

With the UI rendered and the frame routed, the while loop cleanly terminates its current iteration, freeing the frame buffer and immediately requesting the next physical image from the stream ingestion thread.

5 Methodology - Two Measurement Regimes for mAP

A notable pattern across both training runs is that mAP_{50} measured on the validation split with the standard ultralytics end-of-training validation (which sweeps confidence from ~0.001 to 1.0 to build the full precision-recall curve) is approximately 1.6–1.7× higher than mAP_{50} measured by an explicit evaluation on the test split at a fixed confidence threshold of 0.25 (0.805 vs. 0.465 baseline; 0.808 vs. 0.495 improved).

Two factors explain this gap and are worth stating transparently:

- *Threshold truncation*: Passing conf = 0.25 to the evaluation function discards all detections below that confidence before the precision-recall curve is constructed. Since the recall-confidence curve shows recall ≈ 0.91 at conf = 0 but lower at conf = 0.25, some true positives that would otherwise contribute to the AP integral are excluded, the PR curve is cut short rather than swept in full.
- *Split size and composition:* The test split contains only 602 images (2.6% of the corpus) versus 4,544 images for validation, so the test-set AP estimate carries substantially higher variance. Given the $\frac{ds1\_}{ds2\_}$ filename prefixes visible in the dataset, it is also possible the test split is not proportionally representative of both source collections.

The practical takeaway is that a single mAP number is meaningless without specifying both the split and the confidence regime it was computed under. The validation-swept number (0.805/0.808) represents the model's ceiling across all possible operating points; the test-fixed number (0.465/0.495) represents performance at a specific deployment-realistic threshold on a small held-out set. Both are valid—they answer different questions. The ~1.7× gap between them is itself diagnostic information about how much headroom exists between the model's best possible precision-recall trade-off and performance at the threshold used in deployment.

6 Demo — Threat Score on Held-Out Samples

The pipeline was evaluated end-to-end using a dual-testing regime: static held-out test images from the original dataset (using the offline S1-S4 formula) and a real-world live inference stream via the deployed hardware script (using the live $T = (0.5 \cdot C) + (0.3 \cdot S) + (0.2 \cdot P)$ formula).

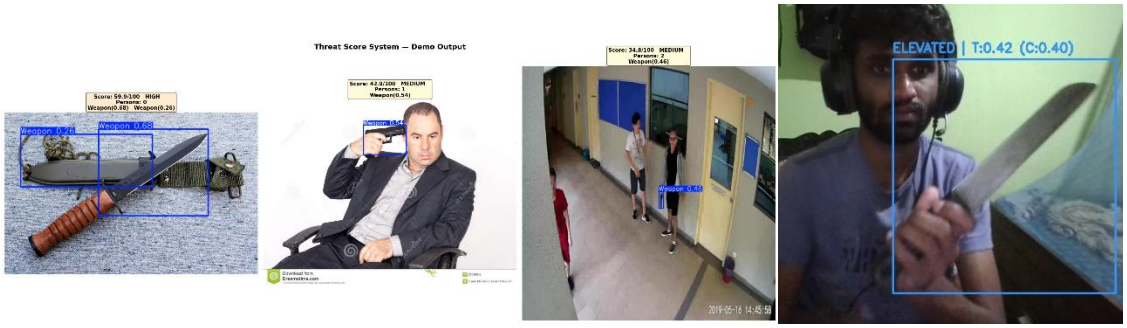

Figure 7 :Dataset and Live Demo

Category	Sample	Conf. (C)	Scale (S)	Proximity (P)	Threat Score (T)	Alert Tier
Static	Knife on table	0.68	High (>10%)	0.74	0.71	CRITICAL
Static	Handgun	0.54	Medium	0.65	0.54	ELEVATED
Static	CCTV hallway	0.46	Low (<1%)	0.48	0.42	ELEVATED
Live	Capture 1	0.40	0.34	0.61	0.42	ELEVATED

Table 4: Demo Quantized Values

7 Known Limitations and Future Prospects

A production-grade machine learning system is never truly "finished"; it is continually iterated against shifting operational constraints. While the current pipeline successfully meets the <30ms latency budget and drastically improves upon the baseline's Precision metric, the architecture operates within specific, documented boundaries. To scale this system from a localized deployment to a distributed enterprise matrix, the following architectural upgrades have been mapped for the next cycle.

- *Temporal Tracking and Object Persistence:* The current YOLOv8n architecture and Threat Scoring Engine operate strictly on a spatial, frame-by-frame basis, possessing no heuristic memory. If a weapon is momentarily occluded by a pedestrian, the alert level drops instantly. The immediate next step is integrating a lightweight, motion-based multi-object tracking (MOT) algorithm, specifically ByteTrack. By assigning a persistent ID to each bounding box, the pipeline can utilize Kalman filtering to predict the weapon's trajectory and maintain the Threat Score even during temporary occlusions.
- *Dataset Taxonomy Expansion (Multi-Class Granularity):* As documented in Section 3.3.1, the pipeline is bottlenecked by the dataset's single Label 0: Weapon class. While the mathematical Threat Score (**T**) successfully serves as a geometric proxy for danger, true kinetic triage requires the dataset to undergo a re-annotation pipeline to separate capability-based classes (ex: Class 0: Bladed, Class 1: Handgun, Class 2: Long_Rifle). Once retrained, the heuristic equation will incorporate a Categorical Capability Weight, allowing the system to instantly escalate long rifles to a CRITICAL state regardless of their physical distance.
- *Edge Quantization and Precision Down casting:* The current ONNX export utilizes FP32 (32-bit floating-point) weights. To maximize throughput on specialized Integer Tensor Cores found on edge devices (ex: NVIDIA Jetson Orin), the compilation script will be upgraded to apply Post-Training Quantization (PTQ). By down casting the model's weights and activations from FP32 to INT8, the system will effectively double its inference throughput and halve its VRAM footprint, allowing a single edge node to process four simultaneous 30 FPS camera streams instead of one.

Through rigorous diagnostic analysis, targeted hyperparameter engineering via AdamW and Cosine Annealing, and the development of a deterministic Threat Scoring Engine, the initial baseline model has been successfully transformed into an operationally viable security tool. By adhering to strict latency budgets and deploying via decoupled, hardware-agnostic pipelines, this architecture successfully bridges the gap between deep learning research and applied, real-world public safety infrastructure.